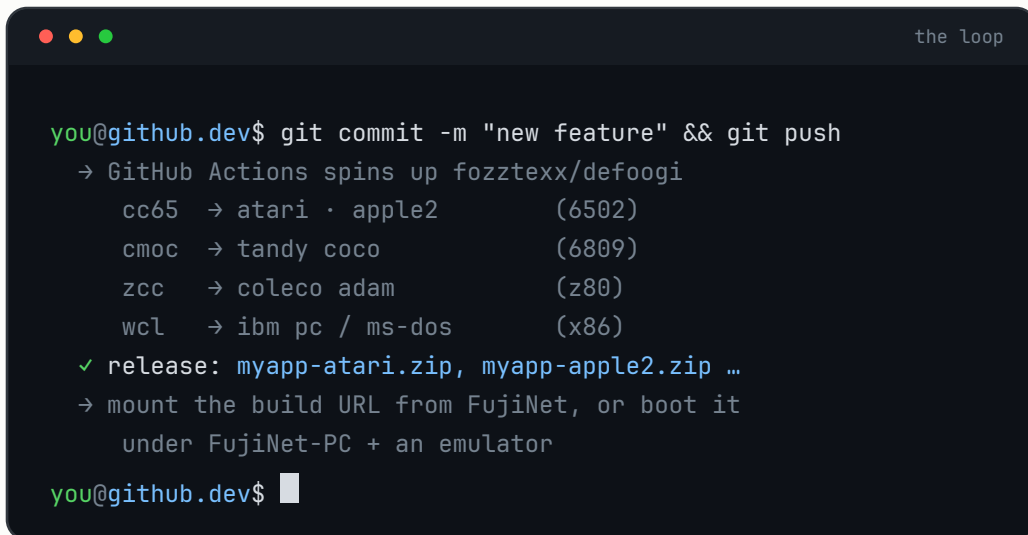


Writing Cross-Platform FujiNet Apps

One C program — Atari, Coleco ADAM, Apple II, Tandy CoCo, IBM PC
— built with fujinet-lib, defoogi, github.dev, and CI/CD.



```
you@github.dev$ git commit -m "new feature" && git push
→ GitHub Actions spins up fozztexx/defoogi
  cc65 → atari · apple2      (6502)
  cmoc → tandu coco         (6809)
  zcc  → coleco adam        (z80)
  wcl  → ibm pc / ms-dos    (x86)
✓ release: myapp-atari.zip, myapp-apple2.zip ...
→ mount the build URL from FujiNet, or boot it
  under FujiNet-PC + an emulator
you@github.dev$
```

Contents

1 • The Workflow at a Glance	3
The five moving parts	3
Why “cross-platform” is realistic here	3
2 • What You Build Against: fujinet-lib	5
The N: device and the device spec	5
The three headers	5
The network lifecycle	5
Error handling	6
JSON without a JSON parser	6
The targets this library supports	7
3 • The App Skeleton	8
One repository, many machines	8
The application Makefile	8
The build targets you get for free	9
How the library gets there	9
4 • Writing the Code	10
The shared core	10
The few machine-specific touches	11
Portability rules of thumb	12
Building it locally	12
5 • Building with defoogi	13
Install it as a command prefix	13
What a build produces, per platform	13
The same image everywhere	14
6 • Editing in github.dev	15
Press the dot	15
Why this matters for retro development	15
7 • CI/CD – Build in the Cloud, Produce Packages	16
The build job	16
Per-platform artifacts (optional)	17
Cutting a Release on a tag	17
8 • Testing on an Emulator with FujiNet-PC	19
The pieces	19
The Atari path, concretely	19
Why bother with emulation at all	20
9 • Testing on Real Hardware	21
1 — Copy to the SD card (WebDAV)	21
2 — Serve it from TNFS	21
3 — Mount the CI build by URL (the magic one)	21
Appendix A • fujinet-lib API Quick Reference	22
fujinet-network.h	22
fujinet-clock.h	22
fujinet-fuji.h (selected)	23
Appendix B • Per-Platform Cheat Sheet	24
Appendix C • Sources & Further Reading	25

This guide shows you how to write **one** C program and have it run on five very different computers — the Atari 8-bit, the Coleco ADAM, the Apple II, the Tandy Color Computer, and the IBM PC under MS-DOS — by talking to the network through FujiNet. You will never touch a 6502 datasheet, install a single cross-compiler, or own all five machines. The whole thing runs as a loop:

THE LOOP

Edit your code in the browser with **github.dev**. **Push**. **GitHub Actions** builds every platform for you inside the **defoogi** container and publishes a downloadable package per machine. **Test** the result on an **emulator driven by FujiNet-PC**, or on **real hardware** — and on real hardware you can have FujiNet **mount the CI build straight off the internet by URL**.

The reason this is possible at all is that FujiNet gives every one of these machines the **same** network device, and `fujinet-lib` gives you the **same** C API to drive it. Your program calls `network_open()`, `network_read()`, `network_json_query()`, `clock_get_time()` — and the library knows how to turn those calls into the right bus protocol for whichever machine it was compiled for.

The five moving parts

Piece	What it does for you
<code>fujinet-lib</code>	A C library with one <code>network/</code> <code>fuji/</code> <code>clock</code> API. You link your app against a prebuilt <code>.lib</code> for each target. (Ch. 2.)
your app + Makefile	One <code>src/</code> tree and a Makefile that just lists the target platforms. A shared build engine does the per-platform work. (Ch. 3–4.)
defoogi	Chris Osborn’s Docker container with every cross-compiler already inside. Builds locally as a command prefix, and is the image CI runs in. (Ch. 5.)
github.dev	The browser version of VS Code. Edit and commit without installing anything. (Ch. 6.)
GitHub Actions	Compiles all platforms in <code>defoogi</code> , uploads a <code>.zip</code> per platform, and cuts a Release on a tag. (Ch. 7.)

Why “cross-platform” is realistic here

It is tempting to assume that targeting a 6502, a 6809, a Z80, and an x86 from a single source file is a fantasy. Three things make it routine for FujiNet apps:

- ▶ **The network device is identical everywhere.** FujiNet presents an `N:` device that speaks HTTP, TCP, UDP, TNFS and more. The protocol detail lives in the firmware, not in your program.
- ▶ `fujinet-lib` **hides the bus.** The Atari reaches FujiNet over SIO, the Apple over SmartPort, the ADAM over AdamNet, the CoCo over a serial link, the PC over RS-232 — but every one of those is behind the same `network_*` functions. You include the same `fujinet-network.h`.
- ▶ **A small portable C subset goes a long way.** Console I/O through `conio.h`, plain `stdio/` `string`, and a handful of `#ifdef`s for the few genuinely machine-specific touches (80-column mode, lower-case, a background colour) is enough for most utilities.

The rest of this book is the concrete how-to: the library API you call (Ch. 2), the project layout and Makefile that turns one source tree into many binaries (Ch. 3), a complete worked app (Ch. 4), building it with defoogi (Ch. 5), editing in the browser (Ch. 6), wiring up CI/CD (Ch. 7), and testing the results under emulation (Ch. 8) and on real iron (Ch. 9).

`fujinet-lib` is the client library you link your application against. It is **not** the FujiNet firmware (that runs on the ESP32 inside the device); it is the small body of host code that runs on the **retro computer** and turns ordinary C function calls into FujiNet bus transactions.

The `N`: device and the device spec

Everything network-shaped goes through a **device spec** string. Its form is:

```
N[unit]:PROTO://[HOSTNAME][:PORT]/PATH...
```

For example `N1:HTTPS://fujinet.online/` or `N2:TCP://192.168.1.10:6502/`. The leading `N` is the FujiNet network device; the optional unit digit (`N1` – `N8`) lets you keep up to eight connections open at once; the protocol is resolved inside the firmware. Protocols available today include `HTTP / HTTPS` , `TCP` , `UDP` , `TNFS` , `FTP` , `SMB` , `SSH` , and `TELNET` .

This is the key abstraction: **your code opens a URL, not a socket**. The firmware owns the TLS stack, the DNS resolver, and the protocol state machine. Your 1.79 MHz machine just reads and writes bytes.

The three headers

The public API is three headers, shipped in every release zip alongside the compiled `.lib` :

Header	Surface
<code>fujinet-network.h</code>	The <code>N</code> : device — open/read/write/close, HTTP verbs and headers, JSON parse/query, filesystem ops, plus error globals.
<code>fujinet-fuji.h</code>	The <code>THE FUJI</code> control device — adapter/Wi-Fi config, host & device (disk) slots, mounting, directory reads, AppKeys, base64, hashing.
<code>fujinet-clock.h</code>	The network clock — <code>clock_get_time()</code> in several binary and ISO string formats, with timezone support.

For a one-page signature dump, see **Appendix A**. The functions you will reach for first are the network ones.

The network lifecycle

Almost every networking task is the same four-beat pattern: **init once**, then **open** → **transfer** → **close** per request.

```
#include "fujinet-network.h"

uint8_t err;

err = network_init();           /* once, at program start      */

network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
n = network_read(url, buffer, sizeof(buffer));
network_close(url);
```

The important signatures (full list in Appendix A):

```
uint8_t network_init(void);
uint8_t network_open (const char *spec, uint8_t mode, uint8_t trans);
int16_t network_read (const char *spec, uint8_t *buf, uint16_t len);
int16_t network_read_nb(const char *spec, uint8_t *buf, uint16_t len); /* non-blocking */
uint8_t network_write(const char *spec, const uint8_t *buf, uint16_t len);
uint8_t network_status(const char *spec, uint16_t *bw, uint8_t *c, uint8_t *err);
uint8_t network_close(const char *spec);
```

`network_read()` returns the number of bytes read, or a **negative** value whose magnitude is the error code — so the idiom is “if it came back negative, `-n` is your error.” `network_open()`’s `mode` is one of the `OPEN_MODE_*` constants and `trans` is an end-of-line translation (`OPEN_TRANS_NONE` , `..._CR` , `..._LF` , `..._CRLF` , `..._PET`):

Constant	Value	Meaning
<code>OPEN_MODE_READ</code> / <code>..._HTTP_GET</code>	<code>0x04</code>	Read / HTTP GET
<code>OPEN_MODE_WRITE</code> / <code>..._HTTP_PUT</code>	<code>0x08</code>	Write / HTTP PUT
<code>OPEN_MODE_HTTP_POST</code>	<code>0x0D</code>	HTTP POST
<code>OPEN_MODE_HTTP_DELETE</code>	<code>0x05</code>	HTTP DELETE
<code>OPEN_MODE_RW</code>	<code>0x0C</code>	Read/write (e.g. TCP)

Error handling

Every fallible call returns a status byte. `FN_ERR_OK` is `0` ; anything else is a problem:

Code	Value	Meaning
<code>FN_ERR_OK</code>	<code>0x00</code>	Success
<code>FN_ERR_IO_ERROR</code>	<code>0x01</code>	I/O problem with the device
<code>FN_ERR_BAD_CMD</code>	<code>0x02</code>	Called with bad arguments
<code>FN_ERR_OFFLINE</code>	<code>0x03</code>	Device / network offline
<code>FN_ERR_WARNING</code>	<code>0x04</code>	Non-fatal, device-specific
<code>FN_ERR_NO_DEVICE</code>	<code>0x05</code>	No network device found
<code>FN_ERR_UNKNOWN</code>	<code>0xFF</code>	Device-specific, unmapped

When you need more detail, the library exposes globals: `fn_device_error` holds the underlying device error after a failed call, and `fn_bytes_read` holds the count from the last read. A typical helper:

```
void handle_err(uint8_t err, char *reason) {
    printf("Error: %d (dev: %d) %s\n", err, fn_device_error, reason);
    cgetc(); /* wait for a key */
    exit(1);
}
```

JSON without a JSON parser

Parsing JSON on a 64 KB machine would be miserable, so FujiNet does it in the firmware. You hand it a URL, ask it to parse the response, then **query** individual fields with a JSONPath-like string:

```

network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE);
network_json_parse(url);
network_json_query(url, "/0/account/display_name", buffer);
network_json_query(url, "/0/content", buffer);
network_close(url);

```

`network_json_query()` returns the length written to `buffer` (negative on error). This is the backbone of nearly every practical FujiNet app — weather, Mastodon, high-score tables, game lobbies — and we build one in Chapter 4.

The targets this library supports

`fujinet-lib`’s own top-level `Makefile` builds these targets:

```
TARGETS = adam apple2 apple2enh atari c64 plus4 vic20 coco msdos pmd85
```

This guide concentrates on the five the question asks for. Each is built by a **different** compiler — which is exactly the plumbing defoogi exists to hide (Ch. 5):

Target	CPU	Compiler	Bus to FujiNet
atari	6502	cc65 (c165)	SIO
apple2 / apple2enh	6502	cc65 (c165)	SmartPort / IWM
adam	Z80	z88dk (zcc , +coleco -subtype=adam)	AdamNet
coco	6809	CMOC (cmoc)	serial (DriveWire-style)
msdos	x86	Open Watcom (wcc)	RS-232

APPLE2 VS APPLE2ENH

`apple2` targets the original II/II+; `apple2enh` targets the enhanced character set. Ship whichever your app needs — most network apps want `apple2enh` for 80 columns.

The fastest way to start is to copy the layout used by the official `fujinet-lib-examples` repository, because its build engine already knows how to download the right library, invoke the right compiler, and build a disk image — for every target — from a single Makefile that does almost nothing.

One repository, many machines

A FujiNet app is a directory tree like this:

```
myapp/
├── Makefile           # names the targets; delegates to the engine
├── application.mk     # optional: extra CFLAGS, unit-test hooks
└── src/
    ├── main.c         # shared code – compiled for every target
    ├── main.h
    ├── common/        # more shared code (recursed)
    ├── atari/         # files compiled ONLY for the atari platform
    ├── apple2/        # ONLY for apple2 / apple2enh
    └── current-target/
        └── coco/      # ONLY for the exact target "coco"
```

The build engine compiles, in order: every `src/*.c` and `src/*.s`; all of `src/common/`; everything under `src/<platform>/` for the current **platform**; and everything under `src/current-target/<target>/` for the exact **target**. That three-level split is your escape hatch: keep the shared logic in `main.c`, and push the rare machine-specific file down into a platform or target folder. Most apps need only `main.c`.

PLATFORM VS TARGET

A **target** is a precise build (`apple2`, `apple2enh`, `atarixl`); a **platform** is the family it belongs to (`apple2`, `atari`). The engine maps targets to platforms (e.g. `apple2enh` → `apple2`, `plus4` → `c64`), so `src/apple2/` is shared by both Apple targets.

The application Makefile

The per-app `Makefile` is boilerplate. You change exactly two lines — `TARGETS` and `PROGRAM` — and delegate everything else to the shared `makefiles/build.mk` engine:

```
# Set the TARGETS and PROGRAM values as required.
TARGETS = atari apple2enh coco adam msdos
PROGRAM := myapp

SUB_TASKS := clean disk test release
.PHONY: all help $(SUB_TASKS)

all:
  @for target in $(TARGETS); do \
    echo "----- Building $$target -----"; \
    $(MAKE) --no-print-directory -f ../../makefiles/build.mk \
      CURRENT_TARGET=$$target PROGRAM=$(PROGRAM) $(MAKECMDGOALS); \
  done
```



```
$(SUB_TASKS): _do_all
$(SUB_TASKS):
    @:
    _do_all: all
```

That loop is the whole trick: for each target it re-invokes the engine with `CURRENT_TARGET` set, so a `make release` becomes “build `myapp` for `atari`, then `apple2enh`, then `coco`, then `adam`, then `msdos`.” To add or drop a machine, edit the `TARGETS` list — nothing else.

WHY TWO DIRECTORIES DEEP

The engine paths are written relative to `../.. /makefiles/`, so your app must live **two levels below** the repo root (e.g. `network/myapp/`), just like the examples (`network/mastodon/`, `clock/get_time/`). Keep that shape and the relative includes resolve cleanly.

The build targets you get for free

From your app folder, the engine gives you:

Command	What it does
<code>make</code>	Compile every target into <code>build/<program>.<target></code> .
<code>make release</code>	Copy each built binary into <code>dist/</code> with its platform suffix.
<code>make disk</code>	<code>release</code> , then wrap each binary in a bootable disk image where a recipe exists (<code>.atr</code> , <code>.po</code> , ...).
<code>make test</code>	Build, then launch the platform’s emulator on the result.
<code>make clean</code>	Remove <code>build/</code> , <code>obj/</code> , <code>dist/</code> .

How the library gets there

You do **not** vendor the library into your repo. The engine’s `makefiles/fujinet-lib.mk` downloads the correct release zip on demand, caches it under `_cache/`, adds its directory to the include path, and links the `.lib`:

```
FUJINET_LIB_VERSION := 4.10.0
FUJINET_LIB_DOWNLOAD_URL = \
    https://github.com/FujiNetWIFI/fujinet-lib/releases/download/\
    v$(FUJINET_LIB_VERSION)/fujinet-lib-$(CURRENT_TARGET)-$(FUJINET_LIB_VERSION).zip
```

The practical consequences:

- ▶ **You pin a version.** Bump `FUJINET_LIB_VERSION` to move; the cache makes rebuilds offline-friendly after the first fetch.
- ▶ **Each target pulls its own zip** — `fujinet-lib-atari-4.10.0.zip`, `fujinet-lib-coco-4.10.0.zip`, and so on — each containing that target’s `.lib` plus the three headers.
- ▶ **CI works the same way**, so a fresh runner downloads the same pinned library you built against locally (Ch. 7).

RESULT

Your repository is tiny: a Makefile, a `src/` folder, and the shared `makefiles/` engine (copied once from the examples repo). Everything else — compilers, the library, disk tools — is supplied by `de-foogi` and by on-demand download.

Let us build a real, useful, genuinely cross-platform app: a **feed reader** that fetches a public Mastodon timeline over HTTPS, parses the JSON in the firmware, and prints the latest post. It is modelled directly on the `network/mastodon` example, generalised to all five machines. The entire program is one `src/main.c`.

The shared core

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <ctype.h>

#include "fujinet-network.h"

uint8_t screen_width;
uint8_t buffer[1024];
char url[] = "n1:https://oldbytes.space/api/v1/timelines/public?limit=1";
char q_name[] = "/0/account/display_name";
char q_created[] = "/0/created_at";
char q_content[] = "/0/content";

void handle_err(uint8_t err, char *reason) {
    printf("Error: %d (dev: %d) %s\n", err, fn_device_error, reason);
    cgetc();
    exit(1);
}

/* Replace anything non-printable so a 40-column ROM font stays sane. */
void filter_buf(void) {
    char *c;
    for (c = (char *)buffer; *c != '\0'; ++c)
        if (!isascii(*c) || !isprint(*c)) *c = '?';
}

void query(char *q) {
    int16_t n = network_json_query(url, q, (char *)buffer);
    if (n < 0) handle_err(-n, "query");
    filter_buf();
}

void main(void) {
    uint8_t err, h;

    setup(); /* per-machine screen setup (below) */
    clrscr();
    cputs("FujiNet feed reader\r\n");

    for (;;) {
        network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE);
```

```

    err = network_json_parse(url);
    if (err != FN_ERR_OK) handle_err(err, "parse");

    query(q_name);    printf("%s", screen_width, buffer);
    query(q_created); printf("%s", screen_width, buffer);
    query(q_content); printf("%s\n", buffer);

    network_close(url);
    cgetc();          /* press a key for the next post */
}
}

```

Read what that does against Chapter 2: open the URL for reading, ask the firmware to parse the JSON, pull three fields by path, print them. There is no HTTP code, no TLS, no JSON tokeniser anywhere in your program. The same source compiles for a 6502 Atari and an x86 PC.

The few machine-specific touches

The only place you need per-machine code is screen setup — 80 columns on the Apple, lowercase, a background colour on the Atari. Wrap it in the compiler's predefined target macros:

```

#include "fujinet-network.h"

#ifdef __APPLE2__
#include <apple2.h>
#endif
#ifdef __ATARI__
#include <atari.h>
#endif

void setup(void) {
    uint8_t height;

#ifdef __APPLE2__
    videomode(VIDEOMODE_80COL);    /* apple2enh */
#endif
#ifdef __ATARI__
    OS.color2 = 0;                  /* black background */
#endif

    screensize(&screen_width, &height);

    if (network_init() != FN_ERR_OK)
        handle_err(FN_ERR_NO_DEVICE, "network_init");
}

```

Macro	Defined when compiling for
<code>__ATARI__</code>	Atari 8-bit (cc65)
<code>__APPLE2__</code>	Apple II (cc65); <code>__APPLE2ENH__</code> also for <code>apple2enh</code>
<code>__CBM__</code> / <code>__C64__</code>	Commodore (cc65)
<code>__COCO__</code>	Tandy CoCo (CMOC)
<code>__MSDOS__</code>	IBM PC / DOS (Open Watcom; the lib's Makefile also defines it)

KEEP THE #IFDEFs SHALLOW

Resist the urge to fork whole functions per machine. The portable subset — `conio.h` (`clrscr` , `cputs` , `cgetc` , `gotoxy` , `screensize`), `stdio` , `string` , `ctype` — covers most needs. Reach for a `#ifdef` only for things that genuinely differ, and keep each one to a line or two.

Portability rules of thumb

- ▶ **Assume 40 columns, light up 80 where you can.** Read the real width with `screensize()` and lay out from that, as the core above does.
- ▶ **The character set is ASCII-ish, not ASCII.** ROM fonts vary; the `filter_buf()` pass keeps a remote feed from spraying garbage. On the Apple II without an enhanced ROM, force lowercase deliberately.
- ▶ **Watch your memory.** These are 64 KB machines. A single 1 KB `buffer` reused across requests (as above) beats many small allocations. Avoid `malloc` churn.
- ▶ **int is 16-bit.** Sizes and counts are `uint16_t` ; read counts come back `int16_t` so they can carry a negative error. Don't assume 32-bit `int` .
- ▶ **No threads, one request at a time** per `N:` unit — but you have eight units (`N1:` – `N8:`) if you genuinely need concurrency.

Building it locally

With the skeleton from Chapter 3 in place and `defoogi` installed (Ch. 5):

```
bash
$ defoogi make clean
$ defoogi make release disk
----- Building atari -----
cc65 src/main.c → build/myapp.atari
dir2atr      → dist/myapp.atr
----- Building apple2enh -----
cc65 src/main.c → build/myapp.apple2enh
mk-bitsy      → dist/myapp-enh.po
----- Building coco / adam / msdos ... -----
✓ artifacts in dist/, owned by you
```

Now let us look at what `defoogi` actually did.

`defoogi` (by Chris Osborn, @fozztexx) is a single Docker container that bundles **every** compiler this guide needs — `cc65` for the 6502 targets, `CMOC` for the CoCo's 6809, `z88dk` for the ADAM's Z80, Open Watcom for the PC's x86 — plus the disk-image tools (`dir2atr` , `AppleCommander` , ...). The companion volume **defoogi Demystified** takes the container apart stage by stage; here we only need to **use** it.

Install it as a command prefix

`defoogi` runs as a prefix in front of your normal build command. Grab the `start` script from the `defoogi` repo, rename it `defoogi` , and put it on your `PATH` :

```
curl -L https://raw.githubusercontent.com/FoZZTexx/defoogi/main/start \
-o ~/bin/defoogi
chmod +x ~/bin/defoogi
```

Then, from your app directory:

```
$ defoogi make release disk # runs the Makefile in-container
$ defoogi cc65 hello.c      # or invoke one tool directly
$ defoogi cmoc program.c
```

The build happens inside the container, but — and this is the feature that makes `defoogi` worth using over a hand-rolled Docker image — **the artifacts land in your working directory owned by you**, not by `root` . No `sudo chown -R` afterwards.

WHAT YOU GET VS. INSTALLING TOOLCHAINS BY HAND

Without `defoogi` you would install `cc65`, `CMOC` (which needs Cygwin on Windows), `z88dk` (built from a specific commit with non-default flags), Open Watcom (a slow two-phase self-hosting build), **and** the disk tools — at mutually compatible versions, on every machine and every CI runner. `defoogi` turns all of that into `docker pull` . See **defoogi Demystified**, ch. 5, for the full “the hard way” matrix.

What a build produces, per platform

`make release` drops a runnable file in `dist/` ; `make disk` additionally wraps it in a bootable medium where the engine has a recipe. The shapes differ by machine:

Target	release output	disk output
atari	<code>myapp.com</code>	<code>myapp.atr</code> (bootable; via <code>dir2atr</code> + <code>picoboot.bin</code>)
apple2 / enh	<code>myapp.apple2</code>	<code>myapp.po</code> (bootable ProDOS; via <code>AppleCommander</code>)
c64	<code>myapp.c64</code> (PRG)	<code>myapp.d64</code>
coco	<code>myapp.coco</code>	no disk recipe in the engine yet — ship the binary
adam	<code>myapp.adam</code>	no disk recipe in the engine yet — ship the binary
msdos	<code>myapp.msos</code>	no disk recipe in the engine yet — ship the <code>.com</code> / <code>.exe</code>

DISK RECIPES ARE PER-PLATFORM AND STILL FILLING IN

The shared engine currently ships `disk` recipes for `atari`, `apple2`, and `c64` (see `makefiles/custom-atari.mk`, `custom-apple2.mk`, `custom-c64.mk`). For CoCo, ADAM, and MS-DOS, `make disk` is effectively `make release` — you get the raw runnable, which you mount or copy directly (Ch. 9). Adding a recipe is just another `custom-<platform>.mk`; the hook (`DISK_TASKS`) is already there.

The same image everywhere

The version string matters. `defoogi` is published to Docker Hub as `fozztexx/defoogi:<tag>` (e.g. `1.4.6`). When you pin that tag, your laptop build and your CI build use byte-for-byte the same compilers — which is the whole reason the cloud build in the next chapters can be trusted to match what you tested locally.

You can write and ship a FujiNet app **without installing anything at all** — not even Docker — by editing in the browser and letting CI do the building.

Press the dot

On any GitHub repository, press the `.` (period) key, or change the URL from `github.com/you/myapp` to `github.dev/you/myapp`. A full VS Code editor opens **in the browser**, with your repo loaded: syntax highlighting, multi-file search, the Source Control panel, extensions.

This is ideal for the inner loop of a FujiNet app, which is mostly editing C and a Makefile:

- ▶ Open `src/main.c`, make a change.
- ▶ Stage it in the Source Control panel, write a commit message, commit.
- ▶ Push (or, since github.dev commits go straight to the repo, your commit **is** the push).

That commit triggers the CI build in Chapter 7, which produces the per-platform packages. You never left the browser tab.

GITHUB.DEV CAN'T BUILD

github.dev is a lightweight **editor**. It has no terminal and cannot run `make` or `defoogi` — there is no compute behind it. That is by design: **editing** happens in github.dev, **building** happens in GitHub Actions (Ch. 7). If you want a browser environment that can actually run a build, that is **Codespaces** — a full container in the cloud, where you **can** `docker run defoogi` or run the toolchains directly. Codespaces costs compute; github.dev is free. For the workflow in this guide you only need github.dev plus Actions.

Why this matters for retro development

The traditional barrier to entry for 8-bit development is the toolchain install. The github.dev + Actions combination removes it entirely: a collaborator with nothing but a web browser can fix a bug in your Atari **and** Apple **and** CoCo build, commit it, and have downloadable disk images a few minutes later. The skills required collapse to “edit C, press commit.”

This is the engine room. A GitHub Actions workflow runs your build **inside the defoogi container**, packages every platform, and publishes the results as downloadable artifacts — and, on a tagged commit, as a GitHub Release. It is the exact pattern `fujinet-lib` uses for its own releases; here it is adapted to build an **application**.

The build job

Drop this in `.github/workflows/ci.yml`. The crucial line is `container:` — every step then runs inside `defoogi`, so `cl65`, `cmoc`, `zcc`, `wcc`, and the disk tools are all simply **there**.

```
name: CI

on:
  pull_request:
    branches: [ main ]
  push:
    tags: [ "v*" ]
  workflow_dispatch:      # lets you run it by hand from the Actions tab

jobs:
  build:
    name: Build all platforms
    runs-on: ubuntu-latest
    container:
      image: fozztexx/defoogi:1.4.6      # the toolchains live here

    outputs:
      files: ${ steps.list_zips.outputs.files }

    steps:
      - uses: actions/checkout@v4

      - name: Build release disks
        run: make release disk          # already inside defoogi

      - name: Package each platform
        working-directory: dist
        run: |
          for f in *; do
            [ -f "$f" ] && zip -j "${f}.zip" "$f"
          done

      - name: List zip files
        id: list_zips
        shell: bash
        run: |
          echo 'files=["'"${( cd dist; echo *.zip ) | \
            sed -e 's/ /"/, "/g'}'""]' >> $GITHUB_OUTPUT

      - name: Upload dist for later jobs
        uses: actions/upload-artifact@v4
```



```

with:
  name: dist
  path: dist/

```

After this job runs on a pull request, the **Artifacts** section of the run holds a downloadable `dist` bundle — your Atari `.atr`, Apple `.po`, and the CoCo/ADAM/PC binaries — built from the commit you just pushed from github.dev. That is the whole “push, wait a moment, download” loop.

RUN INSIDE DEFOOGI, NOT ON THE RUNNER

`container: image: fozztexx/defoogi:<tag>` is what makes this work. The Ubuntu runner has none of these cross-compilers; defoogi has all of them. Pin a real tag (not `latest`) so a green build today stays reproducible tomorrow — the same discipline as pinning `FUJINET_LIB_VERSION`.

Per-platform artifacts (optional)

If you would rather download one platform at a time, fan the list out into a matrix that re-uploads each zip under its own name — the trick `fujinet-lib`’s CI uses:

```

upload:
  needs: build
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'
  strategy:
    matrix:
      file: ${{ fromJson(needs.build.outputs.files) }}
  steps:
    - uses: actions/download-artifact@v4
      with: { name: dist, path: dist }
    - uses: actions/upload-artifact@v4
      with:
        name: ${{ matrix.file }}
        path: dist/${{ matrix.file }}

```

Cutting a Release on a tag

The payoff: when you push a version tag, CI turns the build into a public GitHub **Release** with each platform’s zip attached as a downloadable asset that anyone — including a FujiNet on the other side of the world — can fetch by URL.

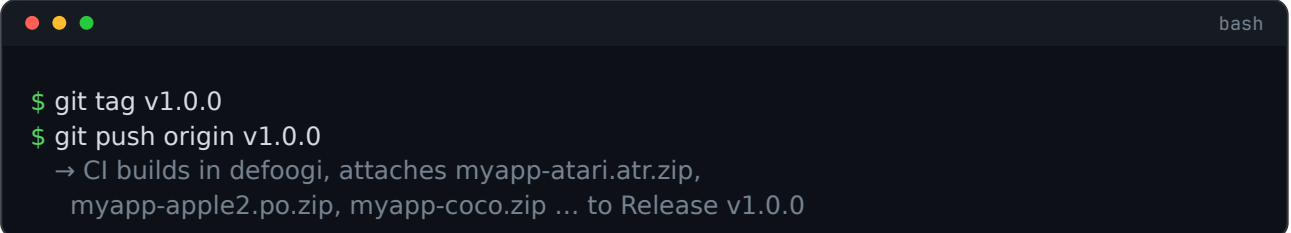
```

tagged-release:
  needs: build
  runs-on: ubuntu-latest
  if: startsWith(github.ref, 'refs/tags/')
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with: { name: dist, path: dist }

    - name: Create the release
      uses: softprops/action-gh-release@v2
      with:
        files: dist/*.zip
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

```

Triggering a release is two commands from anywhere — including the github.dev Source Control panel or a phone:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'bash' is in the top-right corner. The terminal shows two green-prompted commands: '\$ git tag v1.0.0' and '\$ git push origin v1.0.0'. Below the second command is a grey comment line: '→ CI builds in defoogi, attaches myapp-atari.atr.zip, myapp-apple2.po.zip, myapp-coco.zip ... to Release v1.0.0'.

```
$ git tag v1.0.0
$ git push origin v1.0.0
→ CI builds in defoogi, attaches myapp-atari.atr.zip,
  myapp-apple2.po.zip, myapp-coco.zip ... to Release v1.0.0
```

THE ASSET URL IS THE POINT

Each release asset has a stable `https://github.com/you/myapp/releases/download/v1.0.0/...` URL. In Chapter 9 you will hand that URL straight to FujiNet and have it mount the build over the internet — no SD card, no cable. defoogi’s own README calls this out: **“the FujiNet can mount that HTTP asset directly as a disk image.”**

You do not need any of the five machines to test your build. **FujiNet-PC** is the FujiNet firmware compiled to run as a desktop program; pair it with a platform emulator and you have a complete virtual battlestation — the emulated computer on one side, a real FujiNet (in software) on the other, talking to the actual internet.

The pieces

Piece	Role
Platform emulator	Emulates the computer itself (Altirra for Atari; AppleWin / a2 for Apple II; an MSX/Coleco or ADAM emulator; an x86/DOS emulator for the PC).
NetSIO / emulator bridge	Relays the emulated machine's bus to FujiNet-PC over the network. For Atari this is the <code>fujinet-emulator-bridge</code> (a NetSIO hub plus an Altirra custom device).
FujiNet-PC	The firmware as a desktop app — same N: device, web UI, and SD/ host handling as real hardware.

The Atari path, concretely

Atari has the most mature emulator story, and the examples engine wires `make test` to it directly. The flow is:

1. **Start the NetSIO hub** from the bridge repo:

```
cd fujinet-emulator-bridge/fujinet-bridge
python -m netsiohub
```

2. **Point Altirra at it** — add the `netsio.atdevice` custom device (System ► Configure System ► Peripherals ► Devices), and detach `D1:` so FujiNet supplies the disk. Disable **Fast boot** so the device handshake completes.
3. **Connect FujiNet-PC to the hub** — in its web UI, enable **SIO over Network** and give it the hub's host/IP.
4. **Boot your build.** Either let `make test` launch Altirra on your `.atr`, or mount the disk from FujiNet-PC and reboot the emulated Atari with `Shift+F5`.

The examples engine already knows how to start the emulator. With `ALTIRRA_HOME` (or `ATARI800_HOME`) set, from your app folder:

```
bash
$ export ALTIRRA_BIN=~/.altirra/Altirra64.exe
$ defoogi make test      # builds, then launches the emulator
ALTIRRA → build/myapp.atari
✓ your app boots, talks to the real internet via FujiNet-PC
```

`make test` resolves the emulator command from `makefiles/custom-atari.mk` (`ATARI_EMULATOR=ALTIRRA` by default, or `ATARI800`). Each platform that has an emulator exposes its own knobs in the matching `custom-<platform>.mk` ; that file is where you wire up a new one.

COVERAGE IS UNEVEN, AND THAT'S FINE

The turnkey `make test` path is most complete for Atari (and Commodore via VICE). For Apple, ADAM, CoCo, and DOS you may run the emulator by hand and point it at the `dist/` artifact, with FujiNet-PC providing `N:` . The build half of the loop — Chapters 3-7 — is identical for every platform; only the emulator wiring differs.

Why bother with emulation at all

Two reasons. First, **speed**: the edit → build → boot loop is seconds, with no SD card shuffling. Second, **debugging**: emulators like Altirra and AppleWin have real debuggers, and the engine can emit symbol files for them (the Apple `custom-apple2.mk` even generates an AppleWin `debug.scr` from the build's `.lbl`). You can set a breakpoint on `_main` or on a `fujinet-lib` call and single-step your networking code.

Eventually you want it on the actual machine. There are three ways to get a CI build onto real iron, from most manual to most magical.

1 – Copy to the SD card (WebDAV)

FujiNet serves its SD card over WebDAV, so you can push a freshly built disk straight to it from your dev machine using a WebDAV client such as `duck` (Cyberduck CLI):

```
duck --upload dav://anonymous@fujinet.home/dav/myapp.atr \
  dist/myapp.atr -existing overwrite
```

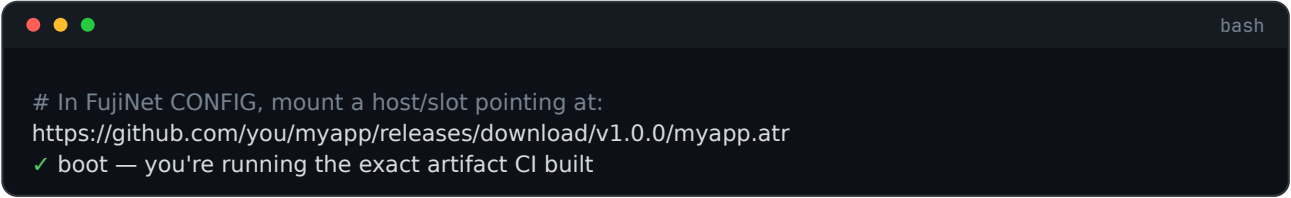
Then boot the FujiNet, pick the disk in CONFIG, and run it. Good for rapid iteration when the machine is on your bench.

2 – Serve it from TNFS

Drop the disk on a TNFS server on your LAN (or a public one) and mount it through CONFIG like any other image. Handy when several people share one build, or the machine isn't next to your computer.

3 – Mount the CI build by URL (the magic one)

This is the move the whole pipeline was built for. Because Chapter 7 published your release asset at a stable HTTPS URL, you can have FujiNet **mount it directly off the internet** — no SD card, no cable, no local server:



```
# In FujiNet CONFIG, mount a host/slot pointing at:
https://github.com/you/myapp/releases/download/v1.0.0/myapp.atr
✓ boot — you're running the exact artifact CI built
```

The complete loop, then, is: **edit in github.dev → push → Actions builds in defoogi and publishes a release → mount the release URL from FujiNet → boot**. You can run code on a 1983 Atari that you wrote, built, and shipped without ever leaving a browser tab. defoogi's README frames this as the headline workflow, and it is: **"Edit and commit your code from any modern machine, wait a moment for the CI build to finish, boot your retro computer, mount the build directly via FujiNet, and run it instantly."**

BOOTING FROM FUJINET, PER PLATFORM

How a machine boots a mounted image is platform-specific — which drive it is, how CONFIG presents slots, which button cold-starts it. Those details live in the per-platform **FujiNet CONFIG / Getting Started** manuals (Atari, ADAM, Apple II, CoCo, MS-DOS) in the `fujinet-manuals` repo. This guide gets the **artifact** to the machine; those guides cover **running** it.

Signatures as of `fujinet-lib` v4.10.0. Full doc comments are in the shipped headers.

`fujinet-network.h`

```
uint8_t network_init(void);
uint8_t network_open(const char *spec, uint8_t mode, uint8_t trans);
uint8_t network_close(const char *spec);
int16_t network_read(const char *spec, uint8_t *buf, uint16_t len);
int16_t network_read_nb(const char *spec, uint8_t *buf, uint16_t len);
uint8_t network_write(const char *spec, const uint8_t *buf, uint16_t len);
uint8_t network_status(const char *spec, uint16_t *bw, uint8_t *c, uint8_t *err);
uint8_t network_ioctl(uint8_t cmd, uint8_t a1, uint8_t a2, const char *spec, ...);

/* HTTP */
uint8_t network_http_set_channel_mode(const char *spec, uint8_t mode);
uint8_t network_http_start_add_headers(const char *spec);
uint8_t network_http_add_header(const char *spec, const char *header);
uint8_t network_http_end_add_headers(const char *spec);
uint8_t network_http_post(const char *spec, const char *data);
uint8_t network_http_post_bin(const char *spec, const uint8_t *data, uint16_t len);
uint8_t network_http_put(const char *spec, const char *data);
uint8_t network_http_delete(const char *spec, uint8_t trans);

/* JSON (parsed in firmware) */
uint8_t network_json_parse(const char *spec);
int16_t network_json_query(const char *spec, const char *query, char *s);

/* Filesystem over N: */
uint8_t network_fs_delete(const char *spec);
uint8_t network_fs_rename(const char *spec);
uint8_t network_fs_mkdir(const char *spec);
uint8_t network_fs_rmdir(const char *spec);
uint8_t network_fs_cd(const char *spec);
uint8_t network_fs_lock(const char *spec);
uint8_t network_fs_unlock(const char *spec);

/* Globals */
extern uint16_t fn_bytes_read;
extern uint8_t fn_device_error;
extern uint16_t fn_network_bw;
extern uint8_t fn_network_conn;
extern uint8_t fn_network_error;
extern uint8_t fn_default_timeout;
```

`fujinet-clock.h`

```
/* TimeFormat values: SIMPLE_BINARY, PRODOS_BINARY, APETIME_BINARY,
   TZ_ISO_STRING, UTC_ISO_STRING, APPLE3_SOS_BINARY */
uint8_t clock_set_tz(const char *tz);
```

```
uint8_t clock_get_tz(char *tz);
uint8_t clock_get_time (uint8_t *time_data, TimeFormat format);
uint8_t clock_get_time_tz(uint8_t *time_data, const char *tz, TimeFormat format);
```

fujinet-fuji.h (selected)

```
bool fuji_get_adapter_config(AdapterConfig *ac);
bool fuji_get_wifi_status(uint8_t *status);
bool fuji_get_host_slots(HostSlot *h, size_t size);
bool fuji_get_device_slots(DeviceSlot *d, size_t size);
bool fuji_mount_host_slot(uint8_t hs);
bool fuji_mount_disk_image(uint8_t ds, uint8_t mode);
bool fuji_open_directory(uint8_t hs, char *path_filter);
bool fuji_read_directory(uint8_t maxlen, uint8_t aux2, char *buffer);
bool fuji_close_directory(void);
bool fuji_read_appkey (uint8_t key_id, uint16_t *count, uint8_t *data);
bool fuji_write_appkey(uint8_t key_id, uint16_t count, uint8_t *data);
/* plus base64_*, hash_*, ssid/scan, boot config ... */
```

APPENDIX B • PER-PLATFORM CHEAT SHEET

Target	CPU	Compiler	release file	disk recipe
atari	6502	cc65 cl65	.com	.atr YES
apple2	6502	cc65 cl65	.apple2	.po YES
apple2enh	6502	cc65 cl65	.apple2enh	.po YES
coco	6809	CMOC cmoc	.coco	binary SOON
adam	Z80	z88dk zcc	.adam	binary SOON
msdos	x86	Open Watcom wcc	.msdos	.com / .exe SOON

YES bootable disk recipe in the shared engine today · SOON ship the runnable; recipe is a future `custom-<platform>.mk`

Target macros: `__ATARI__`, `__APPLE2__` / `__APPLE2ENH__`, `__COCO__`, `__MSDOS__`, `__CBM__` / `__C64__`.

Predefined console subset: `clrscr`, `cputs`, `cputc`, `cgetc`, `gotoxy`, `screenize`, `revers` (`conio.h`).

The two lines you edit in an app Makefile: `TARGETS = ...` and `PROGRAM := ...`.

The version you pin: `FUJINET_LIB_VERSION` in `makefiles/fujinet-lib.mk`, and the `fuzztexx/defoogi:<tag>` image in `ci.yml`.

All verified in the workspace, June 2026.

Repository	What to read
<code>FujiNetWIFI/fujinet-lib</code>	The library, its <code>Makefile</code> / <code>makefiles/</code> , and the public headers (<code>fujinet-network.h</code> , <code>fujinet-fuji.h</code> , <code>fujinet-clock.h</code>). v4.10.0.
<code>FujiNetWIFI/fujinet-lib-examples</code>	The canonical app pattern: <code>makefiles/build.mk</code> , <code>fujinet-lib.mk</code> , <code>custom-*.mk</code> , and worked apps (<code>network/mastodon</code> , <code>network/echo-test</code> , <code>clock/get_time</code>).
<code>FozzTexx/defoogi</code>	The build container — <code>README.md</code> , <code>start</code> , <code>versions.env</code> . v1.4.6.
<code>FujiNetWIFI/fujinet-emulator-bridge</code>	NetSIO hub + Altirra custom device for the FujiNet-PC emulator path.

Companion manuals in `fujinet-manuals/`:

- ▶ **defoogi Demystified** — the container taken apart stage by stage; the per-OS “install it yourself” matrix; CI/CD deep dive.
- ▶ **FujiNet Platform Bring-Up Guide** — for the layer **below** this one: adding a brand-new platform to FujiNet (the ESP32 + RP2350 tandem).
- ▶ The per-platform **FujiNet CONFIG / Getting Started** guides (Atari, ADAM, Apple II, CoCo, MS-DOS) — booting and running images on each machine.